

PROJECT DOCUMENT

LegalEase

AI-Powered Legal Document Generator

Project Type	Artificial Intelligence / Web Application
Frontend	Streamlit
Backend	FastAPI
AI Integration	Google Gemini
Team ID	SWTID-2026-2805
Team Size	4
Team Leader	Riya Ragavi L
Team Members	Raseena S; Rifa Fathima M S; ROOBINI P

Prepared from the supplied LegalEase project documentation

Table of Contents

1. Introduction
2. Project Description
3. Objectives
4. Project Scenarios
5. System Requirements
6. Technology Stack
7. System Architecture
8. Project Workflow and Milestones
9. Core Functionalities
10. Backend API Design
11. Frontend Design
12. Document Formatting and Export
13. User Workflow
14. Deployment and Testing
15. Sample Generated Document
16. Advantages and Applications
17. Limitations and Legal Disclaimer
18. Future Enhancements
19. Conclusion

1. Introduction

LegalEase is an AI-powered legal document generation platform designed to simplify the creation of structured legal documents. The project uses Generative AI together with a user-friendly web interface so that users can provide document-specific information and receive editable legal drafts. The supplied project material describes support for employment contracts, lease agreements, non-disclosure agreements (NDAs), and other agreement types.

2. Project Description

LegalEase provides customizable legal-document templates and generates content according to user inputs such as document type, parties, effective dates, and terms. The application is designed to produce professional-looking documents while supporting editing, branding, and export.

- Users can generate employment contracts, lease agreements, NDAs, and other legal documents.
- Generated content can be previewed and edited before export.
- Documents can be exported as TXT, DOCX, and PDF.
- DOCX and PDF output can include branding elements such as logos, headings, tables, and footers.

- The architecture separates the Streamlit frontend, FastAPI backend, AI generation layer, and formatting utilities.

3. Objectives

- Simplify the creation of structured legal documents through an AI-assisted workflow.
- Collect document requirements through a simple web interface.
- Generate formal document text based on document type, parties, terms, and dates.
- Provide an editable preview so users can modify generated content.
- Support document export in TXT, DOCX, and PDF formats.
- Maintain a modular architecture that can be deployed locally and extended for future cloud deployment.

4. Project Scenarios

Scenario 1 – Employment Contract

A startup founder enters details for a new hire and selects Employment Contract. LegalEase generates an agreement containing areas such as roles, responsibilities, compensation, and confidentiality. The user can customize the wording, add company branding, and download a PDF.

Scenario 2 – Non-Disclosure Agreement

A freelancer enters the parties, confidentiality scope, and effective date to generate a structured NDA.

Scenario 3 – Residential Lease Agreement

A landlord supplies the property address, tenant details, and lease terms. LegalEase creates a formatted lease document with editable sections and DOCX export.

5. System Requirements

5.1 Software Requirements

- Python 3.10 or later
- FastAPI
- Uvicorn
- Streamlit
- Google Generative AI SDK
- python-docx
- FPDF
- Pillow
- Requests
- python-dotenv

5.2 Supplied requirements.txt

The uploaded requirements file lists the following dependencies:

- fastapi
- uvicorn
- python-dotenv

- requests
- streamlit

6. Technology Stack

7. System Architecture

The supplied architecture consists of four principal layers: a Streamlit frontend, a FastAPI backend, a Gemini-based AI core, and document-formatting modules. The frontend collects document details and presents the generated result. The backend exposes the /generate endpoint and routes validated input to the AI generation component. Formatting modules transform generated text into TXT, DOCX, PDF, or HTML preview output.

- Frontend: Streamlit for user interaction, document input, preview, editing, and downloads.
- Backend: FastAPI for API requests and document-generation orchestration.
- AI Core: GeminiDocumentGenerator for structured prompting and generation.
- Formatting Modules: DOCX, PDF, TXT and HTML preview generation.

8. Project Workflow and Milestones

Milestone 1 – Model Selection and Architecture

The project documentation selects Gemini 1.5 Pro and defines the Streamlit + FastAPI + Gemini architecture. The development environment and application structure are also established.

Milestone 2 – Core Functionalities Development

Core legal-document generation, output formatting, editable previews, and downloads are developed. DOCX uses python-docx, PDF uses FPDF, and Streamlit provides the HTML preview.

Milestone 3 – API Logic Integration

FastAPI is initialized with a root endpoint and modular routes. A Pydantic DocumentRequest model validates inputs, while the /generate endpoint invokes GeminiDocumentGenerator.

Milestone 4 – Frontend Development

Streamlit provides fields for document type, parties, terms, and effective date, together with generation, editing, preview, and download functions.

Milestone 5 – Deployment and Testing

The project is prepared for local deployment using a virtual environment, requirements installation, environment variables, Uvicorn for the backend, and Streamlit for the frontend. End-to-end functionality is tested.

9. Core Functionalities

- Legal document generation from user-provided document type, parties, terms, and dates.
- Editable text preview.
- HTML preview of generated legal content.
- TXT export.
- DOCX export with titles, paragraph formatting, logo support, terms table, and footer.
- PDF export with logo, headings, bullet-style terms, header/footer formatting.
- Sanitization of generated text to remove problematic special characters and typographic quotes.

10. Backend API Design

10.1 FastAPI Application

The backend initializes a FastAPI application, defines a root endpoint for service verification, and includes modular routes.

10.2 Document Generation Endpoint

The documented API design includes a POST /generate endpoint.

11. Frontend Design

The Streamlit interface is designed around a simple input-to-document workflow. The supplied documentation describes a three-column layout for centering the logo and a title, followed by input fields for the required legal information.

- Document Type: identifies the document to generate.
- Parties: identifies stakeholders and their roles.
- Terms & Conditions: accepts semicolon-separated clauses.
- Effective Date: identifies when the agreement becomes legally valid.
- Generate Document: sends the request to the backend.
- Edit Document: opens generated content for modification.
- Download: provides TXT, DOCX, and PDF options.

12. Document Formatting and Export

LegalEase supports three main document formats and an HTML preview. The formatting utilities are designed to transform AI-generated text into structured output.

13. User Workflow

- Open the LegalEase Streamlit interface.
- Select or enter the document type.
- Enter the parties involved.
- Enter terms and conditions, using semicolons to separate bullet-style clauses.
- Enter the effective date.
- Select Generate Document.
- Review the generated document in the preview.
- Use Edit Document if changes are required.
- Download the final draft as TXT, DOCX, or PDF.

14. Deployment and Testing

14.1 Local Deployment

- Create and activate a Python virtual environment.
- Install dependencies with `pip install -r requirements.txt`.
- Create a .env file for the Gemini API key.

- Launch the FastAPI backend with Uvicorn.
- Launch the Streamlit frontend using the application file.

14.2 Functional Testing

- Test different document types and combinations of parties, terms, and dates.
- Verify that Gemini generation returns structured content.
- Verify HTML preview rendering.
- Verify document editing.
- Verify TXT, DOCX, and PDF downloads.
- Check logo and footer placement.
- Check bullet formatting and correct embedding of terms.

14.3 Future Deployment Options

The project documentation identifies Render, Railway, Deta, Streamlit Community Cloud, Fly.io, and VPS deployment as possible options for future deployment.

15. Sample Generated Document

The supplied sample legal document is an Employment Contract for Riya as employee and ABC Technologies Pvt. Ltd. as employer, with an effective date of 25/09/2026 and jurisdiction in India. The stated terms are monthly salary, confidentiality, and 30 days' notice for termination. The source explicitly identifies the sample as an AI-generated draft for informational and drafting purposes and recommends review by a qualified legal professional before use.

16. Advantages and Applications

- Reduces the effort required to create a first legal-document draft.
- Provides a guided interface for users without a legal background.
- Supports multiple document types and output formats.
- Allows generated text to be reviewed and edited before export.
- Supports branding through logos and formatted documents.
- Provides a modular architecture that can be extended with additional document types and services.

17. Limitations and Legal Disclaimer

LegalEase is described in the supplied material as an AI-powered drafting platform. Generated documents should not be treated as a substitute for professional legal advice. The supplied sample employment contract explicitly states that it is an AI-generated draft intended for informational and drafting purposes and should be reviewed by a qualified legal professional before use.

The project documentation describes the platform as generating reliable and legally sound content, but the materials provided do not establish independent legal validation of every generated document. Accordingly, human review remains important before a document is used for a real legal purpose.

18. Future Enhancements

- Deeper contract analysis.
- Integration with legal databases.

- Personalized recommendations.
- Broader multilingual capabilities.
- Expanded document templates and legal use cases.
- Further cloud deployment and scalability improvements.

19. Conclusion

LegalEase demonstrates an approach to making legal-document drafting more accessible by combining Generative AI with a user-oriented web interface. The project integrates Streamlit for the frontend, FastAPI for backend API handling, Gemini for AI generation, and document-formatting utilities for TXT, DOCX, PDF, and HTML output. Its workflow covers document input, AI generation, preview, editing, formatting, and download.

The supplied project documentation presents LegalEase as a platform intended to improve accessibility and understanding of legal content. Future extensions identified by the project include deeper contract analysis, legal-database integration, personalized recommendations, and continued expansion of the platform.

Source Basis

This project document was prepared from the uploaded LegalEase project documentation, the supplied requirements.txt file, and the supplied sample legal document. Technical claims and terminology have been kept aligned with those sources.

Key Project Tables

Technology Stack

Layer	Technology	Purpose
Frontend	Streamlit	User input, preview, editing and download interface
Backend	FastAPI	API layer and request handling
Server	Uvicorn	ASGI server for FastAPI
AI Core	Google Gemini	AI-powered legal document generation
Document	python-docx	DOCX formatting and generation
Document	FPDF	PDF generation

Document Formats

Format	Implementation	Features
TXT	Plain text	Simple editable text
DOCX	python-docx	Logo, title, paragraphs, terms table and footer
PDF	FPDF	Logo, headings, bullet-style terms and footer
HTML Preview	Streamlit styling	Styled preview in the application

Sample Legal Document

Field	Sample
Document Type	Employment Contract
Employee	Riya
Employer	ABC Technologies Pvt. Ltd.
Effective Date	25/09/2026
Jurisdiction	India
Terms	Monthly salary; confidentiality; 30 days' termination notice